

MY CLASS RECORDING PLAYLIST:

<https://youtube.com/playlist?list=PLgU-6-TtNPAzpauVSPWpKFbmu5Rbc2dEQ&si=PAzhNHH1uGKTAaTf>

[dijkstra class is missing, giving some supplementary links for that]
For dijkstra algo: [Dijkstra's Shortest Path Algorithm | Graph Theory](#)
[you can ignore the optimization parts and just focus on the simulation for now]

*****GRAPH TRAVERSAL*****

Topic:GRAPH: [6. Graph \[HFN\] Supplementary videos](#)

Basic

1. **Graph basic:** you need to know that in order to do the tasks

Traversal algos [you can refer to my recordings Lecture 1 2]

2. **BFS:** You can find in the class recordings + this slide [6.1 Graph\[BUX slide 8\].pdf](#)
3. **DFS:** Recordings of my class + [6.1 Graph\[BUX slide 8\].pdf](#) this slide

Traversal comparison: [A Comparison of Pathfinding Algorithms](#) [SKIP A* Algorithm for now]

Implementational algos: [refer to my recordings Lecture 2 3]

[Can refer to the slide : [6.1, 6.2, 6.3, 6.4](#) from our shared link:
https://drive.google.com/drive/folders/1EuJTeoAiPXXtdCI5O3k48XDK5_z407xU?usp=drive_link]

1. **BFS:** shortest path in an unweighted graph (refer to the slide above), level order demonstration since it explores level by level so any problem regarding to those, also same slide. For identifying shortest path sequence maintain a parent pointer.
2. **DFS: edge classification:** [DFS - Types of Edges | Edge Classification | Tree Edge, Back Edge, Forward Edge, Cross Edge](#)
How to find Cycle in graph: (DFS method) [Detect cycle in a directed graph](#)
3. **Topological sort: (DFS)** [Topological Sort Algorithm | Graph Theory](#) (BFS/ Kahn's ALgo)
Implementation of top sort: [Shortest/Longest path on a Directed Acyclic Graph \(DAG\) | Graph Theory](#)

4. Strongly Connected Components:

- 1 [Kosaraju Algorithm | Strongly connected components in a graph](#)
- 2 [Tarjans strongly connected components algorithm](#) [OPTIONAL]

*****PATH FINDING/ SHORTEEST PATH ALGO*****

FOR DIJKSTRA SIMULATION FOLLOW THIS VIDEO:

[Graph Data Structure 4. Dijkstra's Shortest Path Algorithm](#)

Single source shortest path algorithms: [USE THESE VIDEOS]

[Dijkstra's Shortest Path Algorithm | Graph Theory](#)

[Ignore the optimization parts]

[Graph Data Structure 4. Dijkstra's Shortest Path Algorithm](#)

Negative weight cycle issue: [Why Dijkstra's Algorithm Doesn't Work with Negative Weights](#)

To handle that, we use: [just know the general idea of bellman]

[Bellman Ford Algorithm | Shortest path & Negative cycles | Graph Theory](#)

*****GREEDY ALGO*****

1. Huffman coding[Greedy Algo]: [refer to my recordings lecture 6]
[7. Huffman Coding](#) [all the links are given in this doc]
2. Fractional Knapsack [not dynamic][Knapsack Problem | Greedy Method | Data structure & Algorithm | Bangla Tutorial](#)

*****MINIMUM SPANNING TREE*****

Minimum Spanning Tree: [6.4 Dijkstra+ MST.pdf](#) [SLIDE]

[refer to my recordings Lecture 5 , 6]

1. Prims Algo: [Prim's Minimum Spanning Tree Algorithm | Graph Theory](#)
2. Kruskal Algo:[Union Find Kruskal's Algorithm](#)
3. Prims vs kruskal:
<https://www.baeldung.com/cs/kruskals-vs-prim-s-algorithm>

*****DYNAMIC PROGRAMMING*****

Dynamic Programming: [refer to my recordings Lecture 7,8]

01 knapsack:

[0/1 Knapsack Problem Dynamic Programming](#)

LCS: [4.9 Longest Common Subsequence \(LCS\) - Recursion and Dynamic Programming](#)

*****PvsNP***** [Just general ideas/intro is needed]

P vs NP problem: [USE THESE VIDEOS+slides]

1. https://bracuacbd-my.sharepoint.com/:f:/g/personal/fakhruddin_gaz_zali_bracu_ac_bd/EkuceTu_illFufQaAcqawGcBfgfdzpODwUFjqGUcBQxnZA?e=0wmLdo
2. [NP.pdf](#)

ALL THE RELEVANT BUX SLIDES ARE GIVEN ALONG WITH THIS DOC IN THIS LINK:

https://drive.google.com/drive/folders/1EuJTeoAiPXXtdCI5O3k48XDK5_z4O7xU?usp=drive_link